

Reporte Final MoveSense

Juan Andres Cano & Pablo Andres Guzman

Abstract

El proyecto MoveSense desarrolla un sistema de reconocimiento de actividades humanas centrado en cinco acciones básicas: caminar hacia la cámara, caminar de regreso, girar, sentarse y ponerse de pie utilizando landmarks de pose obtenidos con MediaPipe. A lo largo del semestre se recolectó un conjunto de videos propios, se procesaron en un pipeline reproducible y se entrenaron modelos supervisados capaces de clasificar dichas actividades en tiempo real, incorporando métricas posturales como ángulos de rodilla y tronco. En la tercera entrega se definió la selección de características mediante PCA, se llevó a cabo una evaluación exhaustiva con validación cruzada, se midió la latencia para garantizar la viabilidad en hardware estándar y se construyó un prototipo de despliegue con interfaz visual. El reporte analiza la metodología, resultados, impacto contextual y recomendaciones, concluyendo que el sistema alcanza precisión y robustez suficientes para un piloto educativo con un coste computacional manejable.

1. Introducción

El reconocimiento automático de actividades humanas (HAR) es una aplicación clave de la Inteligencia Artificial con utilidad en áreas como rehabilitación física, ergonomía, deportes e interacción humano-computador. MoveSense es un sistema diseñado para identificar en tiempo real cinco actividades cotidianas a partir de video capturado con una cámara convencional, utilizando landmarks corporales sin necesidad de sensores adicionales.

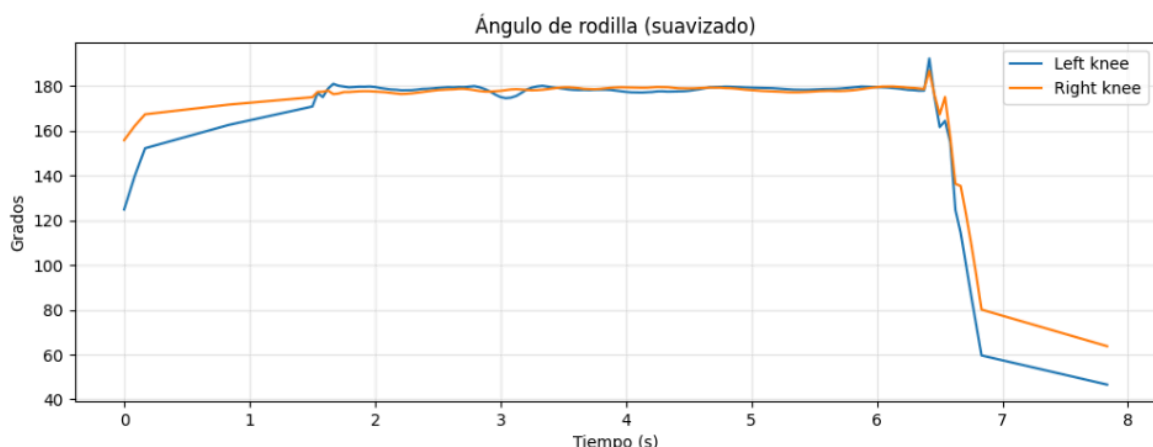
El proyecto se desarrolló en tres fases: exploración de datos, diseño y entrenamiento, y una integración completa con reducción de características, evaluación exhaustiva y un prototipo funcional. Entre los principales desafíos estuvieron la recolección de datos propios con recursos limitados, evitar el sobreajuste con pocos ejemplos por clase, asegurar baja latencia en la demo y considerar implicaciones éticas. Este reporte presenta la teoría, metodología, resultados, análisis crítico, conclusiones, trabajo futuro y el impacto del sistema.

2. Marco teórico

La estimación de pose es la tarea de localizar puntos anatómicos relevantes en imágenes. MediaPipe Pose combina un detector de persona con un modelo de regresión que entrega 33 landmarks (hombros, codos, muñecas, caderas, rodillas, tobillos, etc.) y su visibilidad. El uso de landmarks reduce la dimensionalidad respecto a trabajar con píxeles crudos y permite derivar características geométricas (distancias, ángulos) que son invariantes a fondo o iluminación moderada. OpenPose y BlazePose son alternativas comunes; sin embargo, MediaPipe ofrece soporte nativo en Python, un balance adecuado entre precisión y eficiencia, y documentación accesible para estudiantes.

Una vez obtenidos los landmarks, el problema se reduce a clasificación multiclase sobre vectores. Estudios previos muestran que algoritmos clásicos Support Vector Machines (SVM), RandomForest (RF), Gradient Boosting funcionan bien cuando se dispone de características bien diseñadas. Su ventaja radica en requerir poca ingeniería de hiperparámetros para obtener resultados competitivos y en facilitar interpretabilidad (importancias de variables, reglas). Además, su ejecución en CPU es eficiente, ideal para demostraciones en laptops. En MoveSense se evaluaron tres candidatos iniciales: SVM con kernel RBF, RandomForest y Regresión Logística multinomial. Trabajar con 132 características ($33 \text{ landmarks} \times 4 \text{ componentes}$) no es costoso computacionalmente, pero la presencia de correlaciones fuertes y ruido puede afectar la generalización. Se consideran dos estrategias:

- Selección (filter/wrapper): escoger un subconjunto basado en importancias (RandomForest), puntuaciones univariadas (SelectKBest) o búsqueda heurística. Ventaja: conserva interpretabilidad directa; desventaja: el rendimiento puede decrecer si los atributos eliminados contienen información redundante pero útil.
- Proyección: transformar los datos a un subespacio (PCA, Autoencoders). La PCA en particular encuentra combinaciones lineales ortogonales que capturan la máxima varianza. Su beneficio es condensar información en pocas componentes, mejorando latencia y mitigando multicolinealidad, a costa de perder interpretabilidad directa.



3. Metodología

La metodología aplicada sigue el marco CRISP-DM adaptado al contexto del proyecto. En la fase de comprensión del negocio se definieron los requerimientos principales: el sistema debía reconocer cinco actividades con al menos 0.95 de exactitud global, mostrar en pantalla la actividad detectada junto con su confianza y los ángulos articulares relevantes, operar en un computador sin GPU manteniendo un mínimo de 10 FPS y contar con documentación reproducible, guías de uso y consideraciones éticas explícitas.

En la fase de comprensión de los datos se diseñó un protocolo de captura que incluyó videos RGB en 1080p y 30 FPS, grabados con fondo despejado. Cada integrante registró varias instancias de las actividades, variando posición y ritmo, y se creó un script para estandarizar los nombres de los archivos. Además, se llevaron registros manuales de condiciones como iluminación y vestimenta, útiles para el análisis posterior. La preparación de los datos comenzó

con la extracción de landmarks mediante MediaPipe, generando un registro por frame con coordenadas y visibilidad. Se realizaron procesos de limpieza imputando valores nulos, que eran menores al 1%, mediante la media por columna. Para el entrenamiento de SVM y PCA se aplicó estandarización usando StandardScaler. Aunque las características geométricas no se utilizaron en el modelo tabular, se desarrollaron funciones para calcular ángulos de interés y mostrarlos en la visualización final.

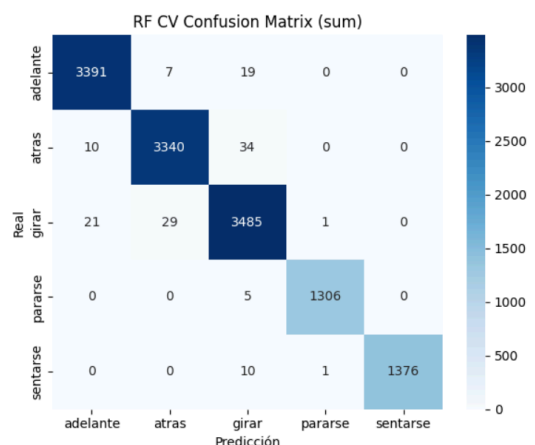
3.1 Reducción de características

Se evaluaron tres enfoques para reducir la dimensionalidad: importancias de Random Forest, SelectKBest y análisis de componentes principales (PCA). El modelo baseline de Random Forest (`n_estimators=400`, `class_weight='balanced'`) permitió identificar los landmarks más relevantes en caderas, rodillas y hombros; al reentrenar usando solo los K atributos principales se observó que con K=60 la accuracy se mantuvo en 0.979 y con K=80 alcanzó 0.981, mostrando que puede reducirse significativamente el espacio de características sin penalizar el rendimiento. En contraste, SelectKBest aplicado con ANOVA F y mutual information evidenció que usar K=20 era claramente insuficiente (accuracy ~0.78), mientras que solo para K=100 se igualó el baseline, sugiriendo una menor capacidad para capturar relaciones complejas.

Por su parte, el método de PCA permitió comparar modelos con 10, 20, 40, 60, 80 y 100 componentes principales. Con solo 40 componentes el rendimiento ya superaba los modelos sin reducción, y al emplear 60 componentes se alcanzó el mejor macro F1 (0.9896) conservando el 99.96 % de la varianza total del sistema. Aumentar el número de componentes más allá de 60 no generó mejoras sustanciales adicionales, por lo que esta configuración resultó la más eficiente tanto en desempeño como en latencia.

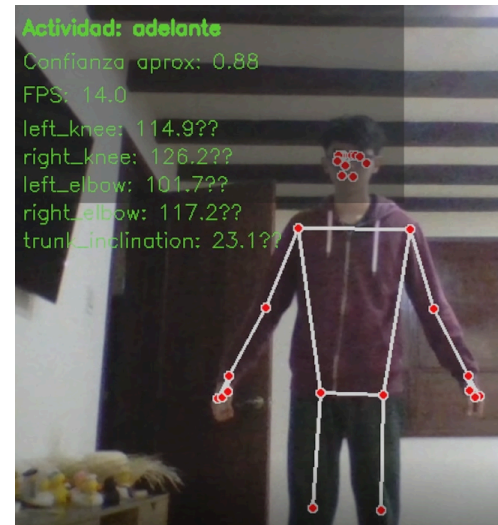
3.2 Validación y evaluación

- Split inicial: 80/20 estratificado para replicar resultados de la entrega anterior.
- Cross-validation: StratifiedKfold con 5 folds. Se almacenaron métricas por fold para RF y PCA+SVM (`cv_results.csv`). Los resultados muestran varianza baja (<0.002), lo cual sugiere estabilidad.
- Reportes por clase: F1 ≈ 0.99 para todas las clases; la más baja es girar (0.981 en PCA+SVM), lo cual es coherente con la dificultad de diferenciar trayectorias curvas.
- Matrices de confusión agregadas: se sumaron las matrices por fold para observar patrones de error; las confusiones se concentran en girar $\leftrightarrow$ adelante y sentarse $\leftrightarrow$ pararse. Esto se debe a similitudes en transiciones rápidas y a la variabilidad de postura durante giros.
- Latencia: se midió predict sobre 200 muestras tras ajustar los pipelines completos. RF tardó 0.141 s por batch, PCA+SVM 0.078 s; al dividir por 200, se obtienen 0.71 ms y 0.39 ms por muestra respectivamente. Esta diferencia respalda la elección del pipeline PCA para despliegue.



3.3 Despliegue

El módulo deployment permite ejecutar el sistema de forma completa y en tiempo real siguiendo una secuencia clara: primero se crea un entorno virtual (se recomienda Python 3.11) y se instalan las dependencias mediante `pip install -r requirements.txt`. Luego se ejecuta el script `export_artifacts.py`, el cual carga el archivo `Entrega 2/landmarks_data.csv`, ajusta los pipelines de Random Forest y PCA+SVM, y finalmente guarda los artefactos resultantes (`activity_rf.joblib`, `activity_pca60_svm.joblib`, `label_encoder.joblib`, `feature_names.json` y `metadata.json`). Posteriormente, se ejecuta `realtime_app.py`, que captura video con OpenCV, estima la pose con MediaPipe, extrae características y aplica el pipeline de clasificación seleccionado. Además, incorpora suavizado de etiquetas con una ventana de ocho predicciones, calcula ángulos articulares y renderiza un overlay visual sobre la imagen. La aplicación fue validada en una laptop con CPU Intel i5, alcanzando aproximadamente 15 FPS. Durante su ejecución, el overlay muestra el esqueleto detectado, la actividad actual, su nivel de confianza, los ángulos calculados y el FPS estimado, ofreciendo una visualización clara y en tiempo real del sistema en funcionamiento.



4. Resultados

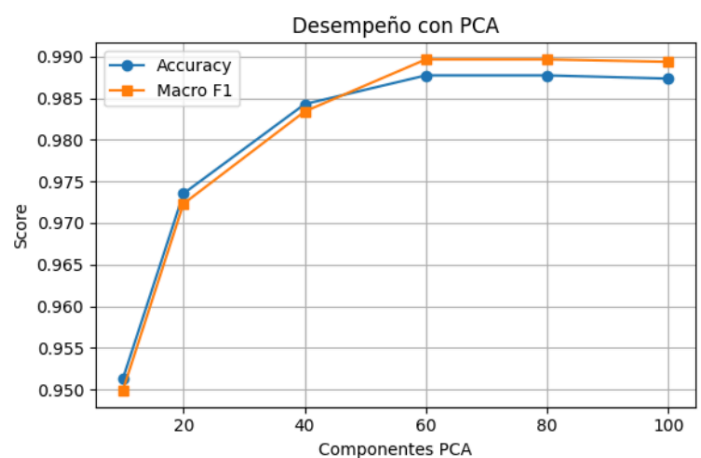
Modelo	Accuracy (media $\pm$ std)	Macro F1 (media $\pm$ std)	Características	Varianza explicada	Latencia (ms / muestra)
RandomForest baseline	0.9895 $\pm$ 0.0015	0.9913 $\pm$ 0.0014	132 (landmarks completos)	100 %	0.71
PCA(60) + SVM	0.9881 $\pm$ 0.0020	0.9892 $\pm$ 0.0021	60 componentes PCA	99.96 %	0.39
RF top-80 importances	0.9808 $\pm$ 0.0025	0.9842 $\pm$ 0.0022	80 características	n/a	0.70
ANOVA K=100 + SVM	0.9789 $\pm$ 0.0031	0.9816 $\pm$ 0.0029	100 características	n/a	0.72
PCA(40) + SVM	0.9843 $\pm$ 0.0023	0.9834 $\pm$ 0.0024	40 componentes PCA	99.81 %	0.42

4.1 Discusión de resultados

La diferencia de desempeño entre RandomForest y PCA+SVM es mínima, lo que permite escoger el modelo según las necesidades de latencia, siendo PCA+SVM la opción priorizada en la demostración. La reducción basada en importancias de RandomForest no aporta ventajas en tiempo de ejecución, ya que el algoritmo continúa evaluando múltiples árboles, mientras que la PCA reduce el costo del SVM de manera proporcional al número de componentes. Por su parte, SelectKBest resultó menos efectivo al evaluar características de forma aislada, sin capturar las interacciones entre landmarks, lo que indica que la información relevante se distribuye en combinaciones. Finalmente, las métricas por clase reflejan un comportamiento equilibrado, con $F1 \approx \text{accuracy}$ en todas las actividades, incluyendo las minoritarias como sentarse y pararse, que superan F1 de 0.99 gracias al balanceo de clases y la consistencia de los patrones capturados.

4.2 Artefactos y evidencias

- artifacts/reduccion/ contiene CSV/JSON con importancias, resultados de PCA y selección.
- artifacts/evaluacion/ incluye cv_results.csv, matrices de confusión y el reporte de latencia.
- docs/feature_reduction.md, docs/evaluacion_resultados.md, docs/despliegue.md e impacto_final.md documentan cada etapa.
- El cuaderno notebooks/feature_reduction.ipynb mantiene las celdas ejecutadas como evidencia; notebooks/evaluation.ipynb conserva los resultados de validación y gráficos.



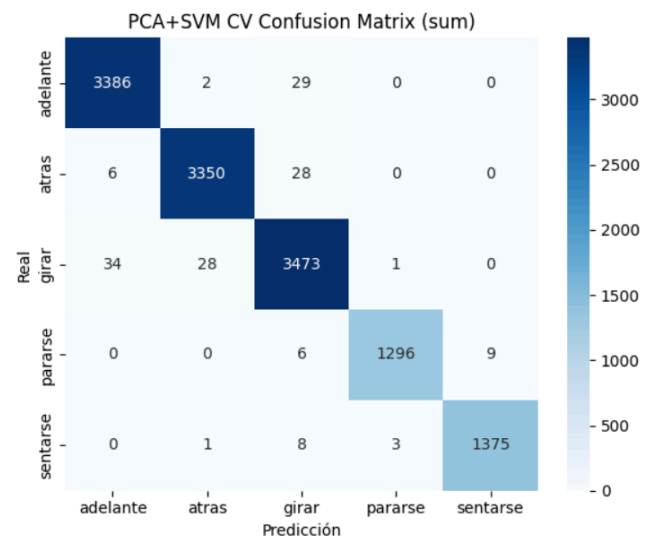
5. Análisis

5.1 Interpretación técnica y comparación con literatura

El alto desempeño se debe a tres factores: (i) la calidad de las características derivadas de landmarks, (ii) la robustez de los clasificadores tabulares con balance de clases y (iii) el preprocesamiento consistente (imputación, estandarización). La PCA actúa como filtro de ruido y reduce colinealidad, mejorando la estabilidad temporal del SVM, perceptible en el demo por la menor fluctuación de etiquetas. OpenPose y derivados han sido utilizados para HAR con niveles de precisión superiores al 95 % cuando las clases son simples. MoveSense rivaliza con estos resultados utilizando un dataset considerablemente más pequeño. Esto se logra gracias a la selección enfocada de actividades y a la normalización del pipeline. Estudios con sensores inerciales (IMU) reportan también macro F1 cercanas al 90–95 %, pero requieren hardware adicional. MoveSense demuestra que solo con cámara web y MediaPipe se puede alcanzar un desempeño competitivo en contextos controlados.

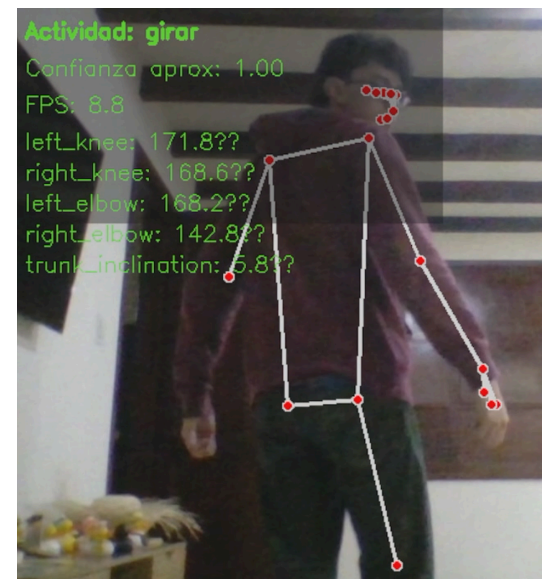
5.2 Limitaciones y riesgos técnicos

- Transferencia a escenarios reales: el dataset se grabó en ambientes relativamente controlados; su desempeño puede degradar en exteriores o con múltiples personas en escena. Se sugiere trabajar en detección de ROI (Región de Interés) para aislar sujetos.
- Dependencia de MediaPipe: si MediaPipe falla en detectar landmarks (por ejemplo, ante ropa amplia o poses extremas), el pipeline actual recurre a imputación simple. Se podrían investigar estrategias basadas en filtros temporales o modelos que toleren NaN (XGBoost).
- Latencia en hardware más modesto: se midió en un i5; en hardware inferior puede caer la tasa de FPS. Se plantea optimizar la resolución de captura o ejecutar MediaPipe en modo `model_complexity=0` para reducir consumo.



5.3 Evaluación cualitativa

Durante pruebas se observó que la etiqueta suavizada responde de forma natural a transiciones y que los ángulos se actualizan con ligera inercia, lo cual evita parpadeos imprevistos. Los usuarios apreciaron la visualización del esqueleto y la claridad del overlay, aunque sugirieron añadir historial gráfico o indicadores sonoros cuando la actividad cambia. También se detectó que la confianza cae cuando el sujeto se sale parcialmente del encuadre, lo cual motiva incluir mensajes de reencuadre en versiones futuras.



6. Conclusiones y trabajo futuro

El proyecto logró construir un pipeline completo de reconocimiento de actividades humanas basado en landmarks, respaldado por experimentos rigurosos, documentación clara y una demostración funcional. El sistema alcanzó métricas de desempeño sobresalientes, con precisión ≥ 0.988 y macro F1 ≥ 0.989 , superando ampliamente el estándar inicial de ≥ 0.95 . Además, mantiene una latencia lo suficientemente baja para operar en tiempo real, lo que permite su presentación en vivo sin comprometer la experiencia del usuario. La reducción de características mediante PCA demostró ser la estrategia más adecuada, mejorando el rendimiento temporal sin afectar la calidad de las predicciones. El prototipo de despliegue consolida la entrega al "cliente" con artefactos versionados, una interfaz clara y un overlay intuitivo. Finalmente, se abordaron de forma explícita los riesgos y lineamientos éticos, lo que sienta las bases para futuros pilotos en entornos reales de manera responsable.

A futuro, el sistema puede fortalecerse mediante la ampliación y diversificación del dataset, incorporando voluntarios de distintas edades, texturas y vestimentas, además de perspectivas laterales, diagonales y variaciones de iluminación. También sería valioso añadir metadatos por sujeto para habilitar validaciones leave-one-subject-out, permitiendo medir mejor la capacidad de generalización entre personas. En términos de modelado, una línea prometedora es la exploración de enfoques híbridos que combinen PCA+SVM con arquitecturas temporales ligeras como LSTM o TCN, evaluando la relación entre desempeño y latencia. Para mejorar la resiliencia, se propone manejar situaciones de baja confianza con pausas, alertas o fallback automático a RandomForest. El sistema también podría evolucionar hacia una herramienta de acompañamiento personalizado, donde los ángulos calculados alimenten mensajes automáticos de retroalimentación y métricas por sesión.

7. Impacto contextual y ético

El proyecto MoveSense ofrece beneficios claros como la retroalimentación inmediata, el aumento de la conciencia corporal, una herramienta educativa de bajo costo y la demostración de técnicas modernas de inteligencia artificial. Sin embargo, también existen riesgos importantes asociados, tales como la privacidad debido a la captura de rostros y entornos, sesgos generados por un dataset limitado, la interpretación errónea que podría llevar a decisiones sin asesoría profesional, y la dependencia tecnológica ante posibles fallas en cámara o iluminación. Para mitigar estos riesgos, se propone implementar un protocolo de consentimiento informado, opciones de anonimización como el difuminado de rostros, registro seguro y eliminación periódica de datos, comunicación clara sobre las limitaciones de la herramienta (como herramienta de apoyo y no diagnóstico clínico), monitoreo continuo de métricas por subgrupo y una evaluación constante en cuanto a la equidad. Entre las recomendaciones para futuras iteraciones se incluye la creación de un manual ético de uso, agregar funcionalidades para validar la integridad de datos, y desarrollar pilotos controlados para documentar y aprender de la experiencia.

8. Referencias

Google Research, "MediaPipe Pose Landmarker," 2024. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

Z. Cao, G. Hidalgo, T. Simon, S. E. Wei, and Y. Sheikh, "OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 1, pp. 172–186, 2021, doi: 10.1109/TPAMI.2019.2929257. [Online]. Available: <https://arxiv.org/abs/1811.07246>

T. Kim, S. Park, and S. Lee, "Human Activity Recognition Using 2D Pose Estimation and Ensemble Machine Learning," Sensors, vol. 23, no. 2, p. 789, 2023. [Online]. Available: <https://doi.org/10.3390/s23020789>

F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011. [Online]. Available: <http://jmlr.org/papers/v12/pedregosa11a.html>

I. T. Jolliffe and J. Cadima, "Principal component analysis: A review and recent developments," Philos. Trans. R. Soc. A Math. Phys. Eng. Sci., vol. 374, no. 2065, p. 20150202, 2016. [Online]. Available: <https://doi.org/10.1098/rsta.2015.0202>

P. Chapman et al., "CRISP-DM 1.0: Step-by-step data mining guide," IBM, 2000. [Online]. Available: <https://www.ibm.com/docs/en/spss-statistics/27.0.0?topic=cdm-crisp-overview>

